

Streamlit Ultimate Cheat Sheet

Interactive Data Apps in Pure Python

1 Getting Started

Installation & Run

```
pip install streamlit
streamlit run app.py
```

Magic Commands Write variables directly to display them.

```
import streamlit as st
import pandas as pd

df = pd.DataFrame({'a': [1, 2, 3]})
df # Draws dataframe automatically
"Hello World" # Draws text
```

2 Display Elements

Text

```
st.title("Main Title")
st.header("Header")
st.subheader("Subheader")
st.text("Fixed width text")
st.markdown("**Markdown** supported")
st.latex(r'e^{i\pi} + 1 = 0')
st.code("print('Hello')", language="python")
```

Data

```
st.dataframe(df) # Interactive
st.table(df) # Static
st.json({'a': 1, 'b': 2})
st.metric("KPI", "42", delta="1.2")
```

3 Input Widgets

Widgets return their value immediately.

```
# Button (Returns True/False)
if st.button("Click Me"):
    st.write("Clicked!")

# Slider
val = st.slider("Select", 0, 100, 50)

# Text Input
name = st.text_input("Name", "John")

# Number Input
age = st.number_input("Age", min_value=0)

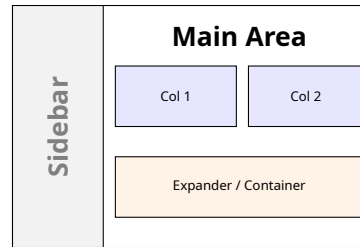
# Selection
option = st.selectbox("Choose", ["A", "B"])
opts = st.multiselect("Pick many", ["A", "B"])
radio = st.radio("Pick one", ["A", "B"])

# File Upload
file = st.file_uploader("Upload CSV", type="csv")
```

```
if file:
    df = pd.read_csv(file)
```

4 Layouts & Containers

Visual Structure



Sidebar

```
with st.sidebar:
    st.write("This is in the sidebar")
    opt = st.selectbox("Filter", ["A", "B"])
```

Columns

```
col1, col2 = st.columns(2)
with col1:
    st.header("Column 1")
    st.image("cat.jpg")

with col2:
    st.header("Column 2")
    st.map(data)
```

Tabs & Expanders

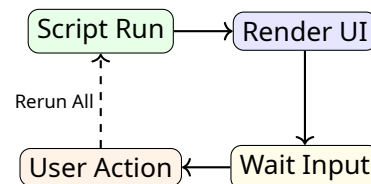
```
tab1, tab2 = st.tabs(["Chart", "Data"])
with tab1:
    st.line_chart(data)

with st.expander("See Details"):
    st.write("Hidden content here...")
```

5 Control Flow

Execution Model

Streamlit reruns the **entire script** from top to bottom whenever a user interacts with a widget.



Forms Batch input to prevent reruns on every change.

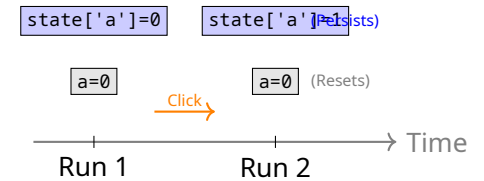
```
with st.form("my_form"):
    col1, col2 = st.columns(2)
    with col1: name = st.text_input("Name")
    with col2: age = st.number_input("Age")

    # Script reruns ONLY when this is clicked
    submitted = st.form_submit_button("Submit")

if submitted:
    st.write(f"Hello {name}, {age}")
```

6 Session State

Persist variables across reruns.



Initialization

```
if 'count' not in st.session_state:
    st.session_state.count = 0
```

Usage & Callbacks

```
def increment():
    st.session_state.count += 1

st.button("Add", on_click=increment)
st.write(f"Count: {st.session_state.count}")
```

7 Native Charts

```
# Simple wrappers around Altair
st.line_chart(df)
st.area_chart(df)
st.bar_chart(df)
st.map(df) # Requires lat/lon columns
```

Third-Party Libs

```
import matplotlib.pyplot as plt
import plotly.express as px

# Matplotlib
fig, ax = plt.subplots()
ax.plot([1, 2, 3])
st.pyplot(fig)

# Plotly
fig = px.scatter(df, x="a", y="b")
st.plotly_chart(fig)
```

8 Caching

Avoid recomputing expensive operations on every rerun.

@st.cache_data For serializable data (DataFrames, JSON, CSV).

```
@st.cache_data
def load_data(url):
    # This runs only once per unique URL
    df = pd.read_csv(url)
```

```
    return df

df = load_data("huge_file.csv")
```

@st.cache_resource For non-serializable objects (DB connections, ML Models).

```
@st.cache_resource
def load_model():
    model = load_huge_model()
```

```
    return model
```

```
model = load_model()
```

Utilities

```
st.spinner("Loading...") # Loading animation
st.balloons()             # Celebration
st.error("Error!")        # Alert box
st.stop()                 # Stop execution
```